

Detecting Earthquakes and Picking P-Wave Arrivals with
Convolutional Neural Networks
Computational Science

Tom Erez
Under the Guidance of:
Shlomo Rozenfeld

2026

Contents

1	Abstract	3
2	Introduction	4
3	Theoretical Basis	5
3.1	Seismology	5
3.2	Signal Processing	6
3.3	Convolutional Neural Networks	7
4	Methodology	8
4.1	Development Environment and Software Stack	8
4.2	Dataset and Preprocessing	8
4.3	Classical Earthquake Classifier	10
4.4	CNN Earthquake Classifier	10
4.5	P-wave Arrival Picker	12
5	Results	13
5.1	Classical Classifier Performance	13
5.2	CNN Classifier Performance	14
5.3	P-wave Picker Performance	15
6	Discussion	16
6.1	Classifier Differences	16
6.2	Picker Use and Effectivity	16
6.3	Project Limitations	16

1 Abstract

Quickly and reliably identifying whether a seismic recording contains an earthquake, and if so, when it begins, is an important task for earthquake early-warning systems and seismological research. In this project I present a CNN 2D earthquake classifier, which is also compared to a statistic based classifier, and a CNN 1D P-wave picker. I trained and tested these neural networks on the Stanford Earthquake Dataset (STEAD). The classifier achieved an F_1 score of 0.998 compared to the classical classifier's F_1 score of 0.904 which is a substantial improvement, while the picker finds P-wave arrivals to an accuracy of 0.337 seconds on average. These results show that even a small CNN operating on compressed spectrograms can reliably classify earthquakes and find their starting point accurately.

2 Introduction

Earthquakes are one of the most destructive natural phenomena, often causing massive damage and loss of life. Detecting and locating them quickly is important for early-warning systems and for seismological research. Manually analyzing signals from a seismogram is slow, so an automated system is needed to perform this task efficiently and accurately.

A seismic signal can be analyzed in many ways and can answer many questions, such as an earthquake's magnitude, depth, or location. Among these, two questions are especially fundamental: is a given recording an earthquake at all and if so, exactly when does it begin? Identifying that starting moment is what lets seismologists later calculate where the earthquake originated and how far away it occurred. Performing these two tasks quickly and reliably across many seismic signals is challenging to do manually, which is exactly the kind of problem machine learning aims to solve.

In this project, I build and evaluate two machine learning systems using the Stanford Earthquake Dataset (STEAD), a large public collection of labeled seismic recordings. The first is a model that classifies a given recording as either an earthquake or background noise. I compare its performance against a classical approach that makes this classification using simple statistical properties of the raw signal. The second is a model that, given a recording known to contain an earthquake, predicts the exact moment at which it begins. Together, these two systems address the two fundamental questions raised above.

3 Theoretical Basis

3.1 Seismology

Seismology is the study of earthquakes and waves that travel through Earth and other planetary bodies. The Earth is made up of tectonic plates, which are massive slabs of rock that make up Earth's outer shell. The edges where these tectonic plates meet are called plate boundaries. There are multiple types of plate boundaries: divergent boundaries (two plates moving away from one another), convergent boundaries (two plates moving towards one another), and transform boundaries (two plates sliding past each other). At plate boundaries, the movement of tectonic plates against one another causes stress to build up in the surrounding rock over long periods of time. When this stress eventually exceeds the strength of the rock, it is released suddenly in the form of a rupture, generating the seismic waves that are felt as an earthquake.

Earthquakes are recorded using instruments called seismometers, which measure the motion of the ground at a particular location over time, producing a trace known as a seismogram. When an earthquake occurs, the energy radiating outward from its source eventually reaches the seismometer, and this is recorded as a change in the signal.

When an earthquake occurs, it produces different types of waves that travel through the Earth and are picked up by seismometers. The two main types are called P-waves and S-waves. P-waves (primary or pressure waves) travel faster and are therefore the first to arrive at a seismometer, while S-waves (secondary waves) travel slower and arrive afterward. Because P-waves always arrive first, the moment a P-wave is detected marks the beginning of an earthquake in a certain place. This is why identifying it is so useful.

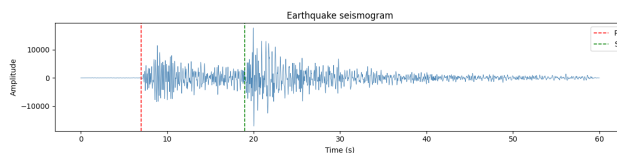


Figure 1: Example of a seismogram showing an earthquake and beginning of waves.

Since P-waves and S-waves travel at different speeds, the time gap between their arrivals at a seismometer can be used to estimate how far away the earthquake occurred. Furthermore, by comparing P-wave arrival times recorded at several different seismometers, it is possible to work out the location of the earthquake itself.

3.2 Signal Processing

Signal processing is a field used across many scientific and engineering disciplines including neuroscience, electrical engineering, and seismology. In general, signal processing includes analyzing a signal in order to gather meaningful data from it, or transforming it into a representation that is easier to work with.

One way to analyze a signal is to summarize it using statistical features such as mean, standard deviation, etc. By reducing an entire signal to a small set of statistics, it is possible to compare signals to one another and to distinguish between different types of signals for example, separating a signal that contains a sudden jump from one that is steady throughout the time series. These statistics can also be used to transform a signal so that recordings can become more directly comparable.

In 1807 Joseph Fourier, a French mathematician and physicist, claimed that any periodic function, such as a signal that repeats over time, can be decomposed into an infinite sum of simple sine and cosine waves and later formalized this in 1822. The **Fourier Transform** is the mathematical tool that puts this idea into practice. Given a signal, it determines exactly which sine and cosine waves and how many of them combine to produce it. The result is a new representation of the signal that shows how much of each frequency it contains. In this way, the Fourier transform allows a signal to be examined not in terms of time, but in terms of the frequencies that make it up.

The problem with performing the Fourier Transform on an entire signal at once is that all information about when each frequency occurs is lost. In order to solve this, the **Short-Time Fourier Transform (STFT)** is used: it splits the signal into short segments of time and performs the Fourier Transform on each segment separately. This way, we obtain the frequency and amplitude of the waves for each individual segment. This preserves information about both when and what frequencies appear in the signal.

After performing the STFT on a signal, we get three dimensions of data: time, frequency, and amplitude. We can turn this into a visual representation called a spectrogram. The x-axis represents time, the y-axis represents frequency, and the color represents amplitude (heat map). In this form, a signal is no longer a single line traced over time, but an image that shows how its frequency and amplitude evolve throughout its duration.

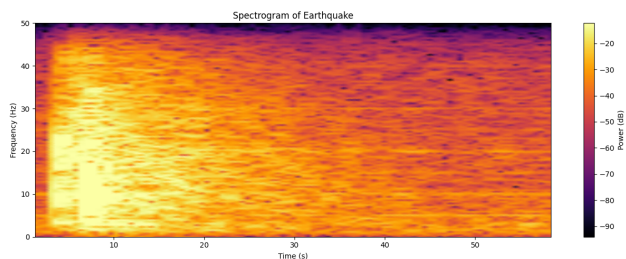


Figure 2: Example of a spectrogram on an earthquake signal.

3.3 Convolutional Neural Networks

Consider the simple drawing below.



I'm sure that if you are able to read this article you can clearly see that this is a drawing of a house. The question is: how do you know?

The answer is that you do not look at the drawing one point at a time. Instead, you recognize shapes within it: a triangular roof, a rectangular door, square windows, straight lines forming walls. Individually, none of these shapes are a house; but when they are formed together your brain immediately identifies that it is a house. This is precisely the idea that a **convolutional neural network (CNN)** is built around. Rather than examining an input one value at a time, it learns to recognize small, local patterns.

To a computer, this drawing of a house is just a massive grid of numbers representing pixel intensities. To replicate how the human brain scans for shapes, a CNN uses small matrices called filters. As this filter moves, it checks if a specific feature is present in that exact spot. Once a filter scans a region, it outputs a raw numerical value representing how strongly a feature matches. However, raw numbers alone aren't enough to build complex visual intelligence. To make sense of these values, the network passes them through an activation function, most commonly the ReLU, which outputs 0 for negative values and leaves positive values unchanged, function. These activation functions allow the network to learn non-linear patterns. A CNN is built on multiple layers of these

When an input is first fed through a freshly initialized network, its parameters are essentially random. Learning is the process by which the network gradually adjusts these parameters so that its guesses become accurate. This is done by comparing the network's output to the correct answer using a **loss function**, a measure of how far off the prediction is. The network then needs to determine how to adjust each of its parameters in order to reduce this loss. This is achieved through an algorithm called **backpropagation**, which works backward through the network, calculating how much each individual parameter contributed to the error. Using this information, the parameters are nudged slightly in the direction that reduces the loss by a process called **gradient descent**. By repeating this cycle, over many examples, the network gradually improves, eventually learning to produce accurate outputs for inputs it was not trained on.

4 Methodology

4.1 Development Environment and Software Stack

Throughout this project I coded with VSCode and utilized Git with GitHub for version control. The project is in the following repository: <https://github.com/t0merez/earthquake-classifier>. I coded with Jupyter Notebooks (.ipynb files) with a Python kernel. I stored all my relevant functions in separate Python files and imported them into my Jupyter Notebooks. Throughout this project, Claude Code was utilized to help write certain parts of the code that will be specified later on.

Throughout the coding of the project I used multiple Python libraries for data processing, signal analysis, and machine learning, including:

h5py: Used to read and extract waveform data and metadata from HDF5 files.

NumPy: Used for vectorized array operations.

Pandas: Enabled efficient management of data.

SciPy: Provided signal processing tools, including the Short-Time Fourier Transform for generating spectrograms and statistical functions such as kurtosis for the classical baseline classifier.

PyTorch: Used to define, train, and evaluate the convolutional neural network models.

scikit-learn: Used to compute classification evaluation metrics.

Matplotlib: Facilitated the generation of training curves, spectrograms, waveform plots, and confusion matrices.

4.2 Dataset and Preprocessing

For this project, I gathered the seismogram recordings from the **STanford EArthquake Dataset (STEAD)**. The dataset is publicly available in the following GitHub repository: <https://github.com/smousavi05/STEAD>.

I used chunk 1, for noise seismograms, and chunk 6, for earthquake seismograms, where each chunk is roughly 14GB of data. Each seismogram included recordings of all three axes (x, y, z); however, I only stored and used the vertical axis (z) as it is the most sensitive to P-wave arrivals. The HDF5 files contain the raw waveforms and the CSV files contain the metadata, and therefore I first converted them into easily fetchable data as *NPY* files. The script which converts the raw files into easily accessible files was written with the help of *Claude Code*.

Afterwards, traces containing NaN values, flat traces (where all values are identical, indicating a dead or unresponsive sensor), and clipped traces (where values exceed a physical threshold, indicating instrument saturation) were removed from the dataset. This ensured that only valid recordings were used.

Each remaining waveform was then normalized using **z-score normalization**, in which every value is rescaled according to:

$$\hat{x} = \frac{x - \mu}{\sigma}$$

where μ and σ are the mean and standard deviation of the waveform respectively. As a result, each normalized waveform had a mean of zero and a standard deviation of one. This was done to ensure that the models were not trained to detect high amplitude signals, since earthquake signals vary widely in amplitude depending on their magnitude and distance from the recording station.

To convert the normalized waveforms into a time-frequency representation, the **Short-Time Fourier Transform (STFT)** was applied to each seismogram. Each recording consists of 6,000 samples captured

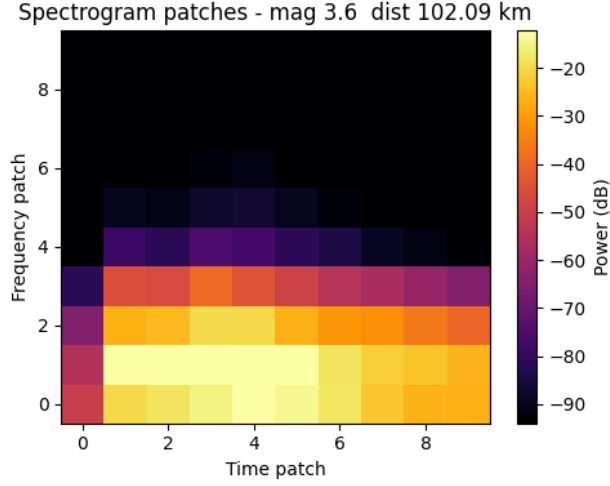


Figure 3: An example of a seismogram that went through the whole process.

at a sampling rate of 100 Hz, giving a total duration of 60 seconds per recording. The STFT was applied using a window size of 256 samples (2.56 seconds) and an overlap of 200 samples (2.0 seconds) between consecutive windows, meaning each window steps forward by 56 samples (0.56 seconds) at a time. The resulting power values were converted to a logarithmic decibel scale.

The output of the STFT is a three-dimensional set of data — time, frequency, and amplitude — which can be visualized as a spectrogram. To reduce its dimensionality to a compact, fixed size suitable for the CNN classifier, the array was divided into a grid of non-overlapping patches and the values within each patch were averaged together, producing a final **10×10 spectrogram grid**.

4.3 Classical Earthquake Classifier

Before training a neural network, I first built a classical classifier to serve as a baseline for comparison. This classifier does not learn from the data in the way a neural network does; instead, it makes its decision by thresholding a single statistical feature of the raw waveform. This classifier’s decision rule is simple enough that it could plausibly be applied by a human inspecting a waveform, making it a natural baseline against which to measure the neural network’s performance.

Two candidate features were considered: the **peak amplitude** of the waveform (the maximum absolute value it reaches) and its **kurtosis** (a statistical measure of how heavy-tailed the distribution of values is — a waveform containing a sudden, sharp burst will have a much higher kurtosis than one consisting of steady noise). To compare them, the distribution of each feature was plotted separately for earthquake and noise recordings in the training set, and the feature that produced the clearest separation between the two classes was selected. After plotting, peak amplitude was chosen, as the noise and earthquake distributions had clearly separated peaks.

The classifier itself works as follows: if the value of the chosen feature exceeds a certain threshold, the recording is classified as an earthquake; otherwise, it is classified as noise. To find the best threshold, 1,000 candidate thresholds were swept across the range of feature values in the training set, and the threshold that produced the highest F1 score was selected. The final classifier was then evaluated on the test set, which it had never seen during the threshold selection process.

4.4 CNN Earthquake Classifier

The earthquake/noise classifier is a two-dimensional convolutional neural network that takes a 10×10 spectrogram (as described in Section 4.2) as input and outputs a single value representing the likelihood that the recording contains an earthquake. Before being used, each spectrogram was normalized using the mean and standard deviation computed from the training set, ensuring that the validation and test sets were processed without any information leaking from them into the normalization statistics.

The classifier is a small convolutional neural network (CNN), summarised in Table 1. It consists of three convolutional blocks followed by a fully connected classification head. Each convolutional block applies a 3×3 convolution, followed by batch normalisation (which rescales activations to stabilise training) and a ReLU activation function. The second block ends with a 2×2 max-pooling layer, which keeps only the largest value in each 2×2 patch and so halves the spatial resolution. The third block ends with an adaptive average-pooling layer that reduces the feature maps to a fixed 2×2 size. The resulting $128 \times 2 \times 2$ feature map (512 values) is flattened and passed through a fully connected layer with 256 units and a ReLU activation. A dropout layer (rate 0.4) randomly disables 40% of these units during training to reduce overfitting. A final linear layer maps the 256 values to a single output. A sigmoid function applied to this gives the predicted probability that the input window contains an earthquake signal; a threshold of 0.5 is used to obtain a binary prediction.

Table 1: CNN architecture. Output shapes are given as (channels, height, width).

Layer	Description	Output shape
Input	10×10 spectrogram	(1, 10, 10)
Block 1	Conv 3×3 , BatchNorm, ReLU	(32, 10, 10)
Block 2	Conv 3×3 , BatchNorm, ReLU, MaxPool 2×2	(64, 5, 5)
Block 3	Conv 3×3 , BatchNorm, ReLU, AdaptiveAvgPool	(128, 2, 2)
Flatten	—	(512)
FC1	Linear, ReLU, Dropout (0.4)	(256)
FC2	Linear	(1)

The dataset is split into training (60%), validation (15%) and test (25%) sets. Earthquake recordings are split at the level of seismic events, so that all recordings of the same earthquake fall into the same set; this prevents the network from being tested on an event it has effectively already seen during training. Noise recordings, which do not belong to any event, are split at random.

The network is trained to minimise the binary cross-entropy loss function, which is derived from the likelihood function (Bernoulli):

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^N [y_i \log(\hat{p}_i) + (1 - y_i) \log(1 - \hat{p}_i)], \quad (1)$$

where $y_i \in \{0, 1\}$ is the true label (noise or earthquake) and $\hat{p}_i$ is the predicted probability for example i .

The network’s weights are updated using an adaptive gradient-based optimisation method (Adam), which automatically adjusts the size of each update based on how the loss has been changing, allowing the model to learn efficiently without the step size having to be tuned by hand. A small penalty is also applied to large weight values during training, which, similar to dropout, discourages the network from relying too heavily on any single connection and helps reduce overfitting.

Training runs for up to 30 passes through the full training set. If the validation loss does not improve for three consecutive epochs, the size of the weight updates is halved, allowing the model to make finer adjustments as training progresses. Training is stopped early if the validation F_1 score has not improved for five consecutive epochs, and the version of the model that achieved the best validation F_1 score is kept as the final model.

Note: Claude Code was utilized to plan and optimize minor parts of this neural network.

4.5 P-wave Arrival Picker

The P-wave arrival picker is a one-dimensional convolutional neural network that takes a full 60-second waveform (6000 samples) as input and outputs a single value: the predicted sample index at which the P-wave arrives. Unlike the earthquake/noise classifier, this is a regression task rather than a classification task, and the network is trained only on earthquake recordings.

The picker is a small 1D convolutional neural network, summarised in Table 2. It consists of three convolutional blocks followed by a single linear output layer. Each block applies a convolution along the time axis, followed by a ReLU activation and a max-pooling layer that reduces the sequence length by a factor of 4. After the three blocks, the original 6000-sample waveform has been reduced to 64 channels of length 93. Rather than averaging each channel down to a single value, as in the classifier, the output of the last block is flattened directly into a vector of $64 \times 93 = 5952$ values; this preserves information about where along the waveform the activity occurs, which is needed to pinpoint an arrival time. This vector is passed through a single linear layer that outputs one number: the predicted sample index of the P-wave arrival.

Table 2: Picker CNN architecture. Output shapes are given as (channels, length).

Layer	Description	Output shape
Input	Waveform	(1, 6000)
Block 1	Conv (kernel 7), ReLU, MaxPool 4	(16, 1500)
Block 2	Conv (kernel 7), ReLU, MaxPool 4	(32, 375)
Block 3	Conv (kernel 5), ReLU, MaxPool 4	(64, 93)
Flatten	–	(5952)
FC	Linear	(1)

The picker is trained only on earthquake recordings that have an annotated P-wave arrival; noise recordings and earthquakes without an annotation are excluded. The same train/validation/test split described above is used, so recordings from the same earthquake event do not appear in more than one set.

Since the goal is to predict a continuous arrival time rather than a class label, the network is trained to minimise the mean absolute error (MAE) between the predicted and true arrival sample:

$$\mathcal{L} = \frac{1}{N} \sum_{i=1}^N |\hat{t}_i - t_i|, \quad (2)$$

where t_i is the true P-wave arrival sample for example i and $\hat{t}_i$ is the network’s prediction (1 sample corresponds to 0.01 s at the 100 Hz sampling rate). This is the same quantity used to evaluate the picker’s performance.

As with the classifier, the network’s weights are updated using the Adam optimization method, with batches of 256 examples. Training runs for up to 30 epochs and stops early if the MAE validation has not improved for seven consecutive epochs.

Note: Claude Code was utilized to plan and optimize minor parts of this neural network.

5 Results

5.1 Classical Classifier Performance

The threshold for the classical baseline was selected by sweeping over 1000 candidate values of the peak amplitude $\max |x|$ on the training set and choosing the value that gave the highest F_1 score. This selected a threshold that corresponded to a training F_1 score of 0.900.

On the test set, this threshold gives an F_1 score of 0.904, close to the training value, indicating that the threshold generalises well to unseen data. Figure 4 shows the resulting confusion matrix: 93.3% of earthquakes in the test set were correctly identified, with the remaining 6.7% missed, while 88.1% of noise windows were correctly classified as noise, with the remaining 11.9% incorrectly flagged as earthquakes.

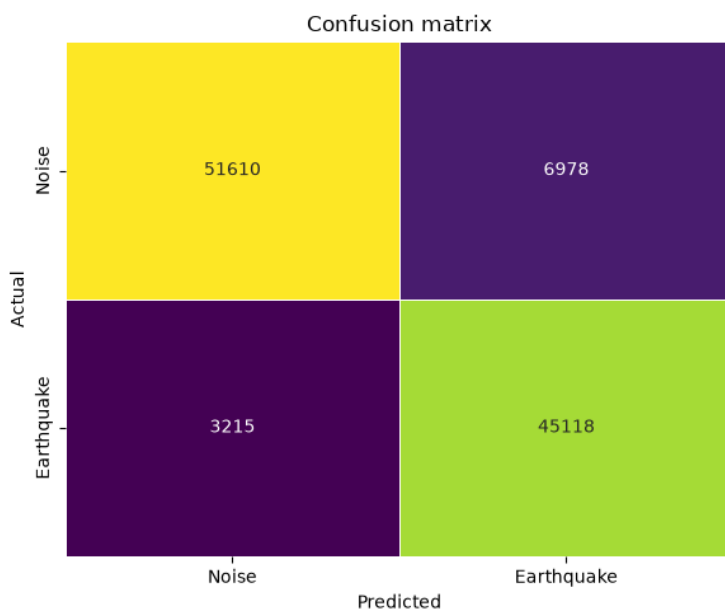


Figure 4: Confusion matrix for the classical (peak-amplitude threshold) classifier on the test set.

This shows that a simple amplitude-based rule without a neural network already performs reasonably well, correctly classifying around 90% of the test set.

5.2 CNN Classifier Performance

The CNN classifier was trained for up to 30 epochs, with the version of the model achieving the highest validation F_1 score kept as the final model. Training stopped early after 29 epochs once validation performance stopped improving, with a best validation F_1 score of 0.998.

On the test set, the CNN achieves an F_1 score of 0.998. Figure 5 shows the resulting confusion matrix: 99.7% of earthquakes in the test set were correctly identified, with the remaining 0.3% missed, while 99.8% of noise windows were correctly classified as noise, with the remaining 0.2% incorrectly flagged as earthquakes.

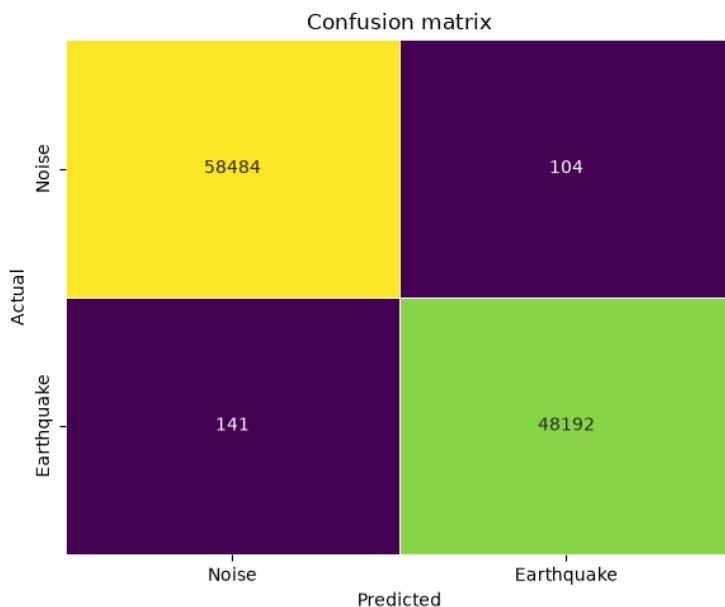


Figure 5: Confusion matrix for the CNN classifier on the test set.

This shows that the CNN almost perfectly separates earthquakes from noise on this dataset, correctly classifying over 99.7% of examples in both classes. This is a substantial improvement over the classical baseline’s 90%.

5.3 P-wave Picker Performance

The picker was trained for 15 epochs, with the version of the model achieving the lowest validation mean absolute error (MAE) kept as the final model. The best validation MAE was 0.336s, reached at the final epoch.

On the test set, the picker achieves an MAE of 0.337s (33.7 samples at the 100Hz sampling rate). Figure 6 shows the distribution of residuals (predicted minus true arrival sample) on the test set: most predictions fall close to zero, indicating that the picked arrival time is, on average, within a fraction of a second of the true value.

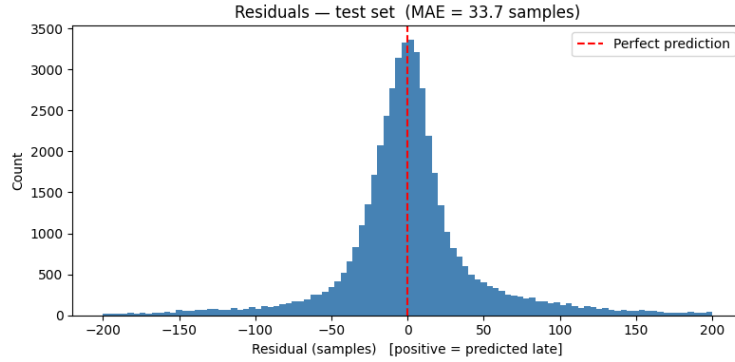


Figure 6: Distribution of residuals (predicted minus true P-wave arrival sample) for the picker on the test set.

This shows that the picker can localise the P-wave arrival to within roughly a third of a second on average.

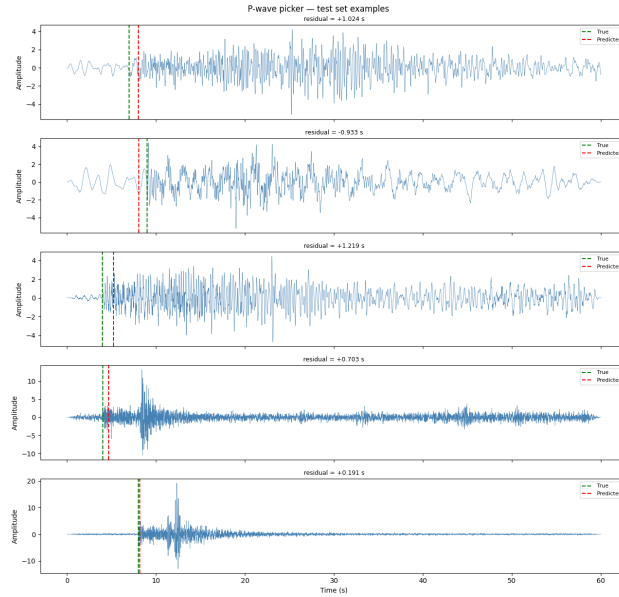


Figure 7: Example of P-wave picks from test set. (Green is the true arrival and red is the predicted arrival.)

6 Discussion

6.1 Classifier Differences

The most striking result of this project is the gap between the two classifiers: the classical peak-amplitude threshold reaches an F_1 score of 0.904, while the CNN reaches 0.998. The reason for this gap comes down to how much information each method is given to work with. The classical classifier reduces an entire 60-second recording down to a single number, the peak amplitude, and compares this one number to a threshold. The CNN, on the other hand, is given the full 10×10 spectrogram, which describes how the signal’s energy is spread out across both time and frequency.

This extra information matters because peak amplitude alone does not always distinguish an earthquake from noise. A sudden, sharp spike in the recording, for example from a passing vehicle, wind, or a seismogram glitch, can reach the same peak amplitude as a real earthquake. An earthquake produces a burst of energy across many frequencies as the P-wave and then the S-wave arrive, while a short noise spike tends to be much shorter and concentrated in a different part of the frequency range. The classical classifier has no way of telling these two cases apart, since both can end up with a similar peak amplitude. The CNN, however, can learn to recognize the particular pattern an earthquake leaves behind in a spectrogram.

6.2 Picker Use and Effectivity

The whole reason picking the P-wave arrival matters is that the gap between the P-wave and S-wave arrival times is used to estimate how far away an earthquake occurred, and comparing arrival times across multiple stations is used to find its location. In general, a P-S gap is several seconds for a nearby earthquake, so an error of about a third of a second in picking the P-wave arrival is fairly small in comparison, and likely would not change a resulting distance or location estimate by very much. This suggests the picker’s predictions could realistically be used as an input to find the location of an earthquake.

That said, an MAE is an average, and Figure 6 shows that this does not tell the whole story. While most predictions fall very close to zero, a small number of recordings have residuals of a second or more in either direction. If a few stations all produced a pick like this for the same earthquake, the error in the resulting location estimate could end up being noticeably larger than the average MAE alone would suggest.

6.3 Project Limitations

There are also a few limitations to this project that are worth being upfront about, mostly coming from choices made to keep the project manageable.

The first is that only the vertical (Z) component of each recording was used. This made sense since the Z component is the most sensitive to P-wave arrivals, which both systems depend on. However, S-waves tend to show up more strongly on the horizontal components, so it is possible that including them could have given the models extra useful information, particularly the classifier.

The second is that the spectrogram given to the CNN classifier was compressed down to a 10×10 grid, which throws away a lot of the detail produced by the STFT. Given how well the CNN performed anyway, this amount of detail seems to have been enough to separate the two classes, but it is possible that a finer spectrogram could help with some of the harder cases, at the cost of a larger and slower model.

Finally, both models were trained and tested on data from STEAD, recorded by a particular set of seismic stations. Although the train/test split was done carefully so that the same earthquake never appears in both sets, it is still possible that the models have learned something specific to the way STEAD recordings look, and that they would perform differently on recordings from other networks or instruments.

References

- [1] Rozenfield, S. *Computational Science: Assignments and Reference Implementations*. Hemda, Tel Aviv, 2026. Internal course materials and personal communication.
- [2] S. M. Mousavi, Y. Sheng, W. Zhu, and G. C. Beroza, “STanford EArthquake Dataset (STEAD): A Global Data Set of Seismic Signals for AI“, *IEEE Access*, vol. 7, pp. 179464–179476, 2019.
- [3] Zhu, W. and Beroza, G. C. PhaseNet: A deep-neural-network-based seismic arrival-time picking method. *Geophysical Journal International*, 216(1):261–273, 2019.